

V0088

Advanced Information Security

高级信息安全

Instructor: **Prof. Xiaohong Yu**

Graduate School of Information and Communication Technology,
Ajou University, Korea

操作系统安全

- 操作系统安全边界
- 隔离、内存保护与访问控制
- DAC与MAC
- 可信路径 (Trusted Path) 与可信计算基 (TCB)
- 现代操作系统的安全设计

操作系统安全

- 密码学
 - 身份验证
 - 访问控制
 - 网络安全
 - 软件安全
 - ➔ 操作系统安全
-
- 操作系统是所有安全机制最终执行的位置。

操作系统安全

- 操作系统下安装的同样的软件。
- 影响因素：
 - 权限配置
 - 补丁更新
 - 系统**隔离**能力
 - 安全策略
 - 操作系统设计



案例： WannaCry勒索病毒

- WannaCry为什么危害巨大？
 - 利用SMB（服务器消息块）漏洞传播
 - 获得系统权限
 - 加密用户文件
 - 横向传播



攻击Windows系统的SMBv1协议漏洞，获取最高系统权限

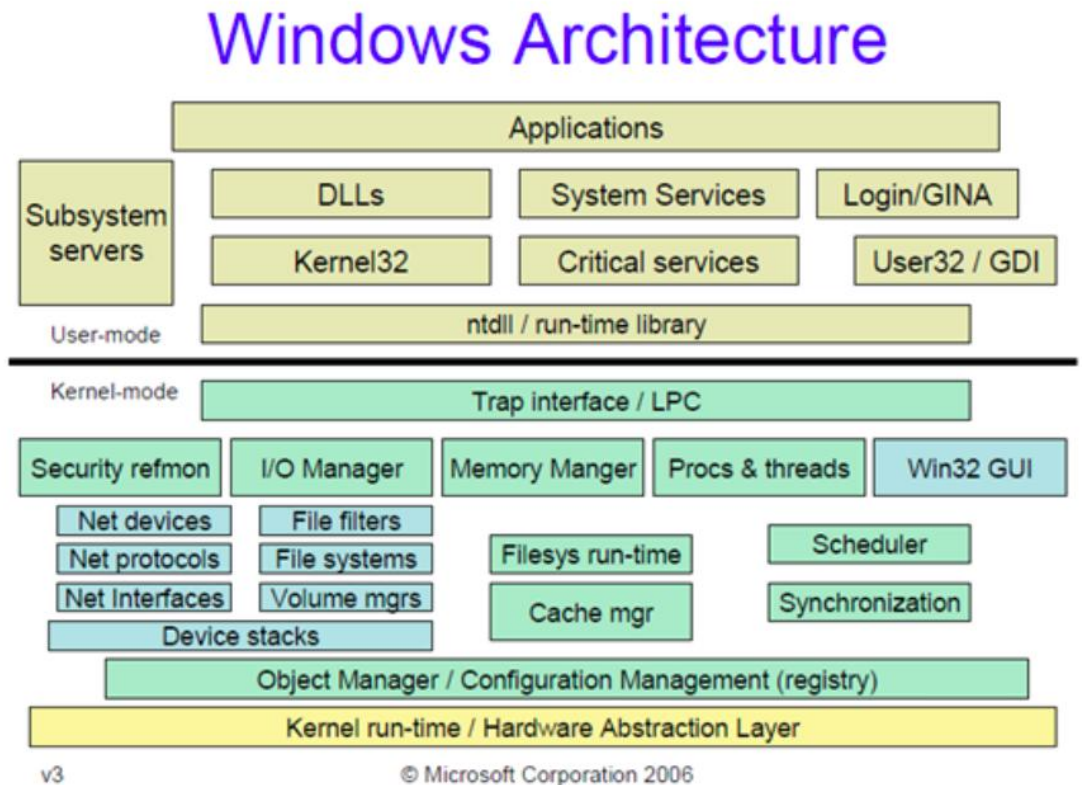
利用NSA的“永恒之蓝”（EternalBlue）漏洞利用程序透过互联网对全球运行Microsoft Windows操作系统的计算机进行攻击的加密型勒索软件兼蠕虫病毒

- 病毒为什么能够修改系统中的关键文件？

操作系统安全

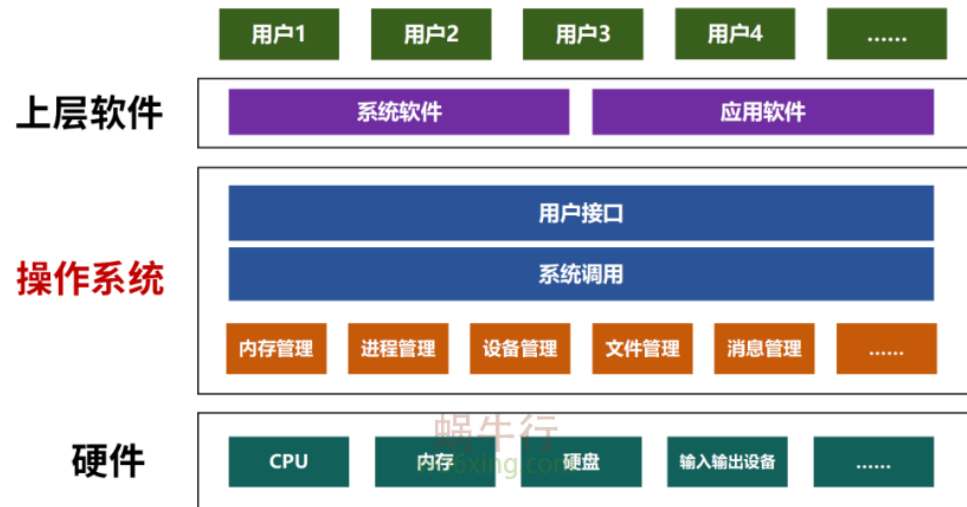
- 安全的操作系统必须能够有效应付各种关键安全问题。无论是偶然事故还是恶意攻击。

- 保证系统可信运行
- 防止非法访问
- 控制资源使用
- 防止恶意代码破坏



操作系统安全

- 核心目标:
 - 隔离 (Isolation)
 - 内存保护 (Memory Protection)
 - 访问控制 (Access Control)



<https://www.wo6xing.com/os/what-is-75-ppt.html>

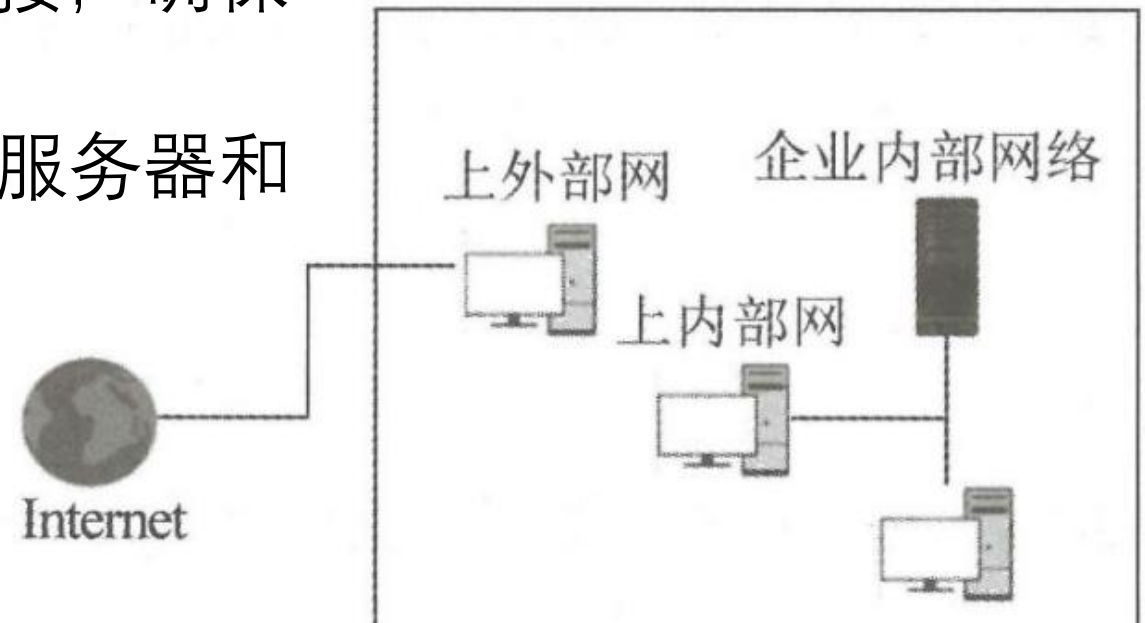
操作系统安全-隔离

- 操作系统最基本的安全问题就是**隔离**。程序之间互不影响。
- 操作系统必须使用用户之间彼此隔离，如同进程隔离一样。
- 没有隔离：
 - 一个程序读取其他程序数据
 - 一个程序种植其他程序
 - 恶意软件快速扩散
- 有隔离：
 - 每个程序运行在独立环境。
 - 故障不会扩散。

□隔离可分为：物理隔离，时间隔离，逻辑隔离和密码隔离

物理隔离

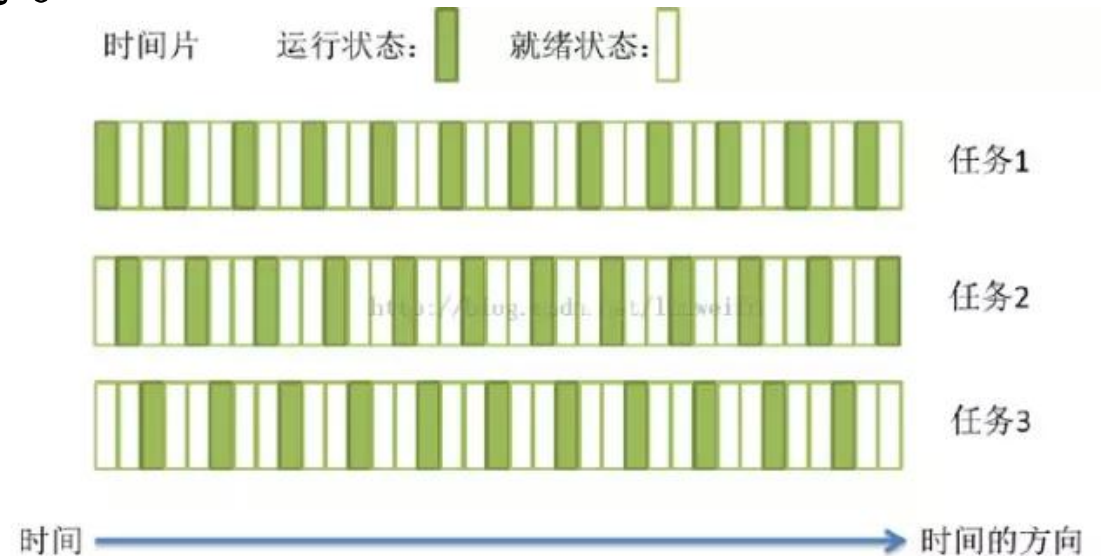
- 物理隔离：通过实体硬件断开连接，确保系统或网络在物理上互不相通。
- e.g., 直接拔出网线，使用独立的服务器和存储设备



- 常见应用：涉密内网、银行核心交易系统、关键基础设施控制系统。

时间隔离

- 时间隔离：在多任务或多进程共享同一个处理器时，严格限制各个任务对计算时间的占用，确保任何单个任务的执行延迟或执行超时都不会影响其他任务的正常运行。

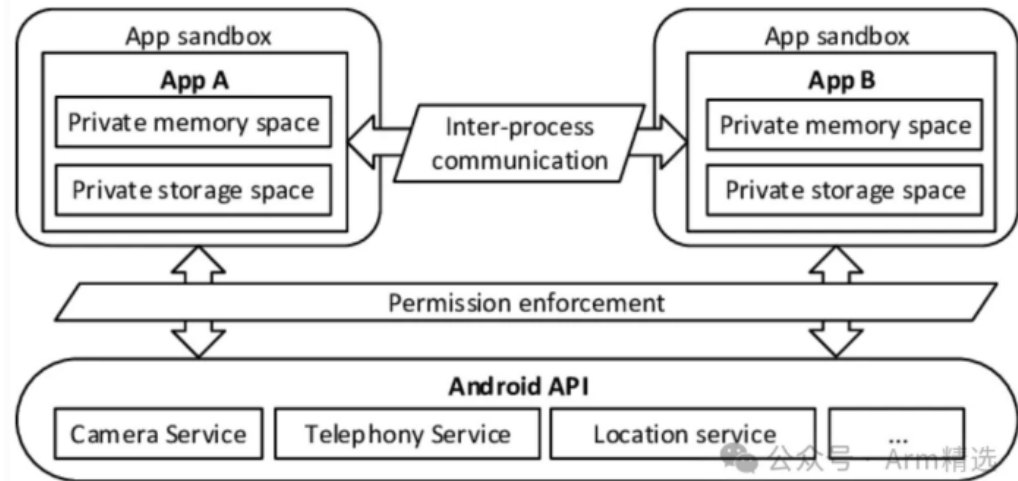
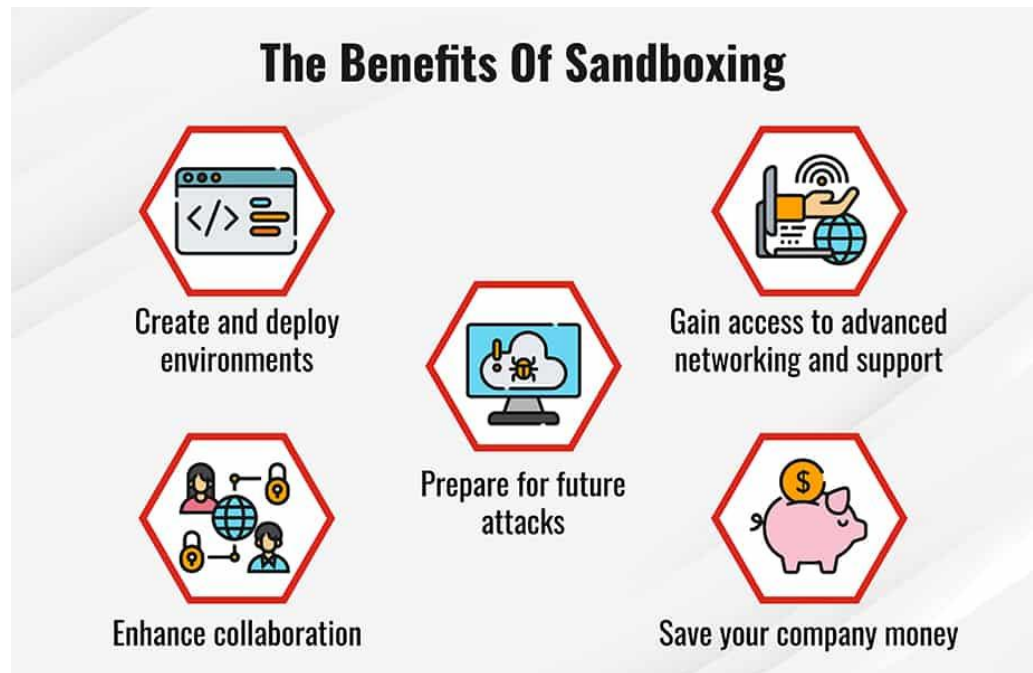


- 常见应用：实时操作系统（RTOS）
容器与云计算
虚拟化（Hypervisor）

<https://www.cnblogs.com/wujing-hubei/p/5968082.html>

逻辑隔离

- 逻辑隔离：通过沙箱的方式来实施，各个进程拥有自己的沙箱。
- 利用软件指令、内存管理单元（MMU）和权限控制，在共享同一套物理硬件的环境下，将不同进程、用户或虚拟机分割开。
- 应用：多租户云平台
沙盒与安全浏览
移动端应用隔离



逻辑隔离

- 进程隔离：
 - 每个进程拥有独立地址空间。
- 特点：
 - 进程A无法直接访问进程B
 - 崩溃互不影响
 - 提高稳定性与安全性

Google Chrome (23)

Google Chrome

Google Chrome

Google Chrome

Google Chrome

Google Chrome

Google Chrome

Google Chrome

Google Chrome

Google Chrome

3.0%

1.1%

0.2%

0.7%

0%

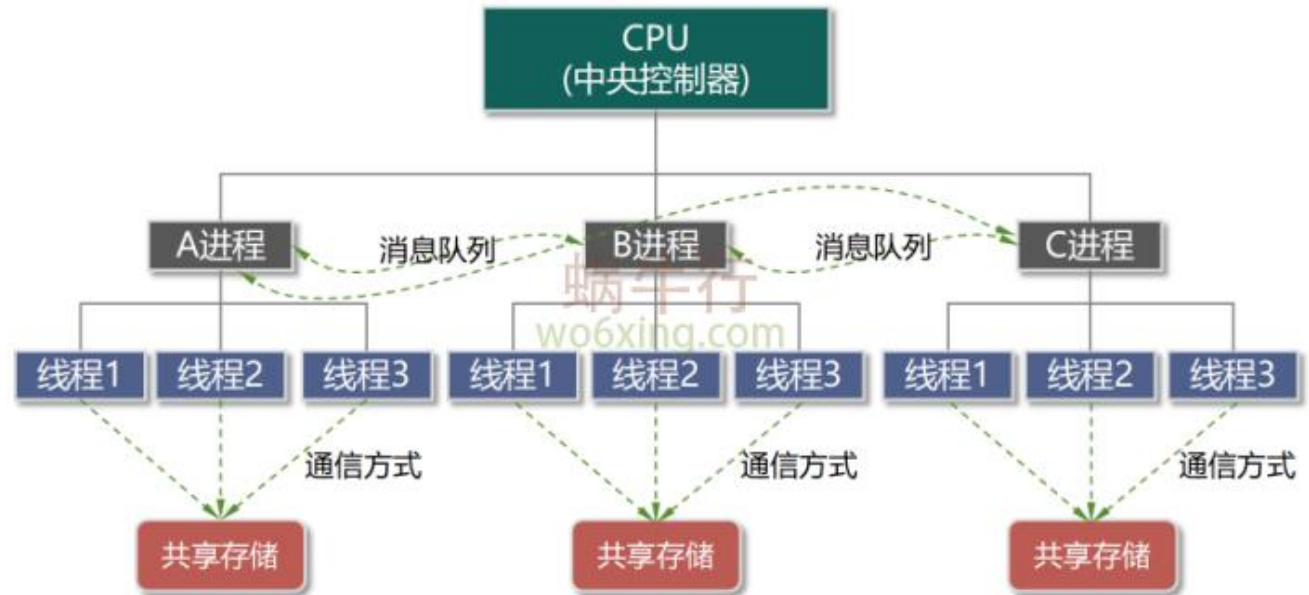
0.3%

0.5%

0%

0%

0%



密码隔离

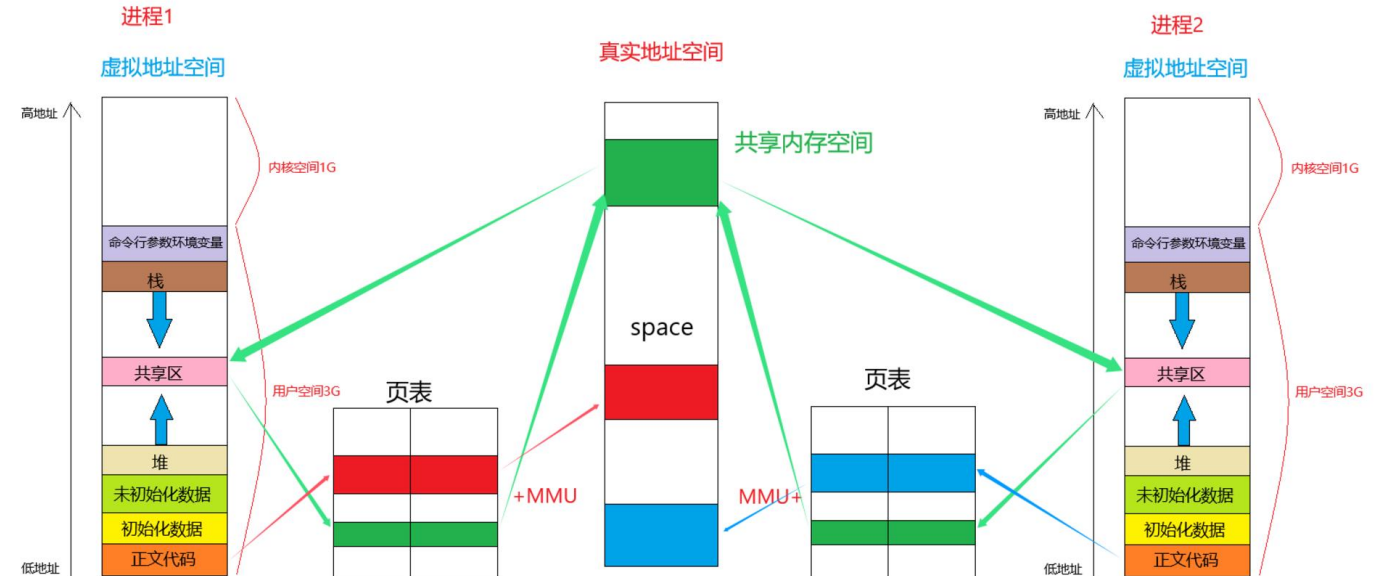
- 密码隔离：通过**加密隔离**使得外来用户无法理解真实的消息。
- 防止不同用户相互获取认证信息。
- 核心：
 - 用户密码不可见
 - 系统管理员也不直接看到明文密码
 - 密码必须隔离存储



内存保护

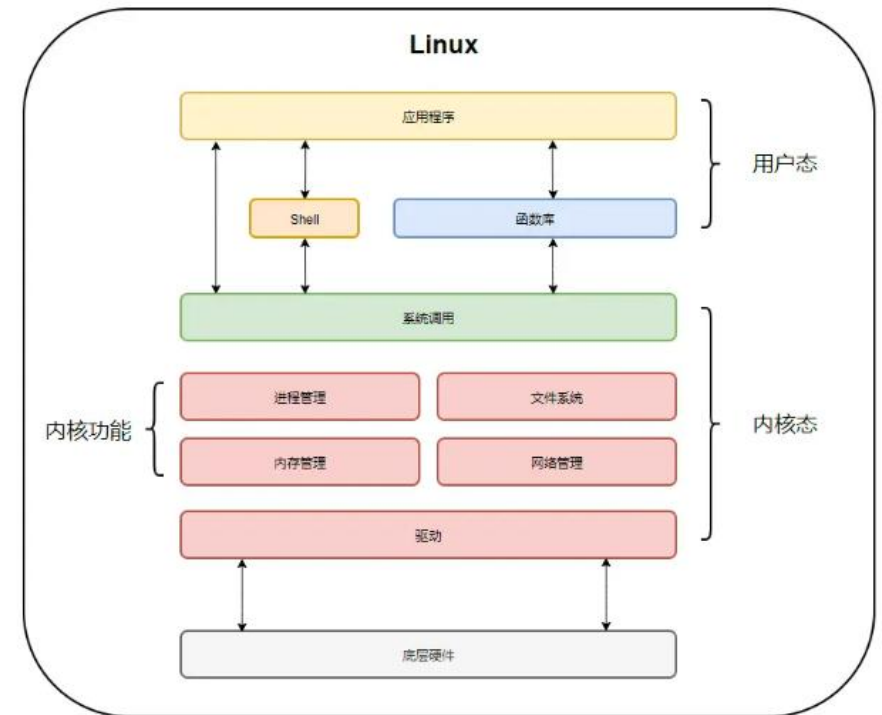
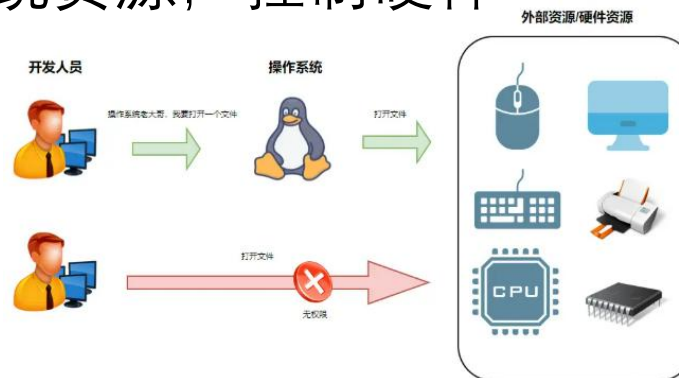
- 操作系统除了进行隔离防护之外，还需要进行内存保护。
- 栅栏地址可以用来进行内存保护，栅栏是指一块用户及其进程无法跨越的特殊地址，只有操作系统可以对栅栏的一段进行操作。

- 如果没有内存保护：
 - 程序覆盖其他程序内存
 - 信息泄露
 - 系统崩溃
 - 提权攻击



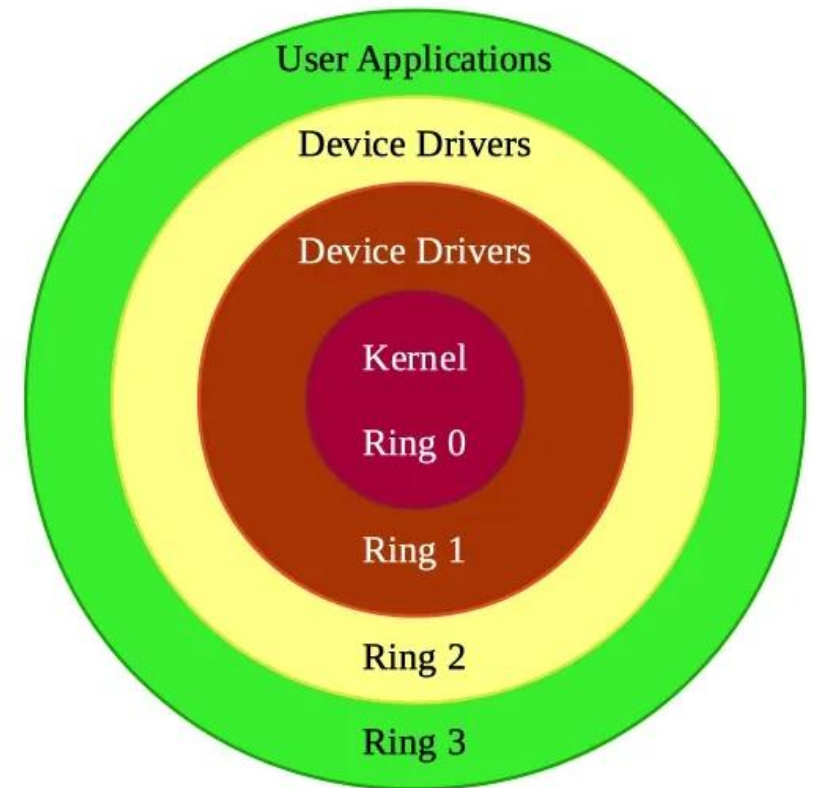
内存保护-用户态和内核态

- 内存保护机制依托于CPU的**特权级**，将操作系统的运行环境严格划分为内核态和用户态。
- **用户态**：
 - 权限低，普通程序运行，无法直接访问硬件
- **内核态**：
 - 权限最高，管理系统资源，控制硬件



内存保护-权限模型

- Ring 权限模型：CPU硬件层面的安全隔离机制将系统资源划分为4个特权级别Ring0到Ring3。限制程序对内存、硬件和敏感指令的访问。
 - Ring0：最高权限。操作系统内核运行层，执行CPU指令，访问物理内存、I/O设备及系统资源。
 - Ring1/2：中间权限。用于设备驱动程序或服务。现代操作系统通常将驱动放到Ring0。
 - Ring3：最低权限。常规应用程序运行在此层。无法直接访问硬件或内核数据。必须通过**系统调用**。

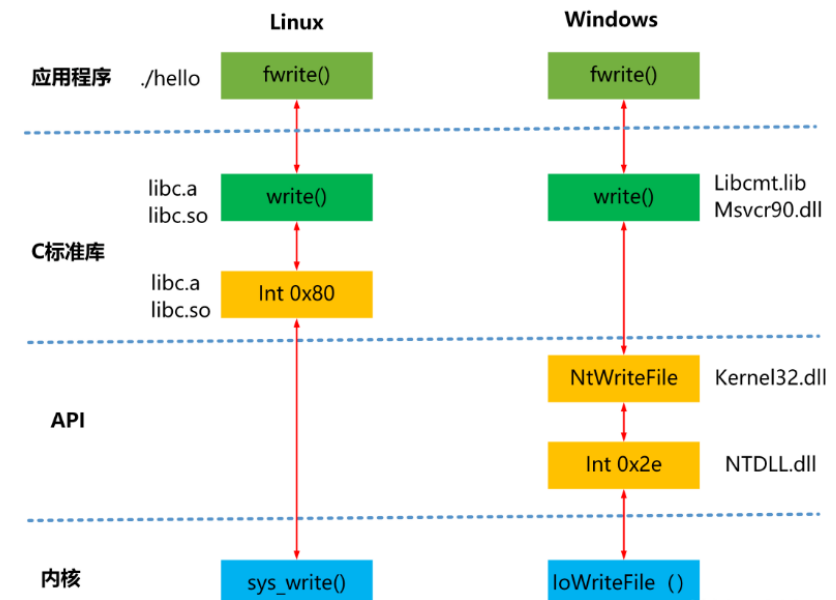


内存保护-系统调用

- 系统调用：从用户态到内核态申请使用系统资源。从而让用户程序安全得请求操作系统服务。
- System Call：用户程序请求内核（Kernel）代办事务的接口

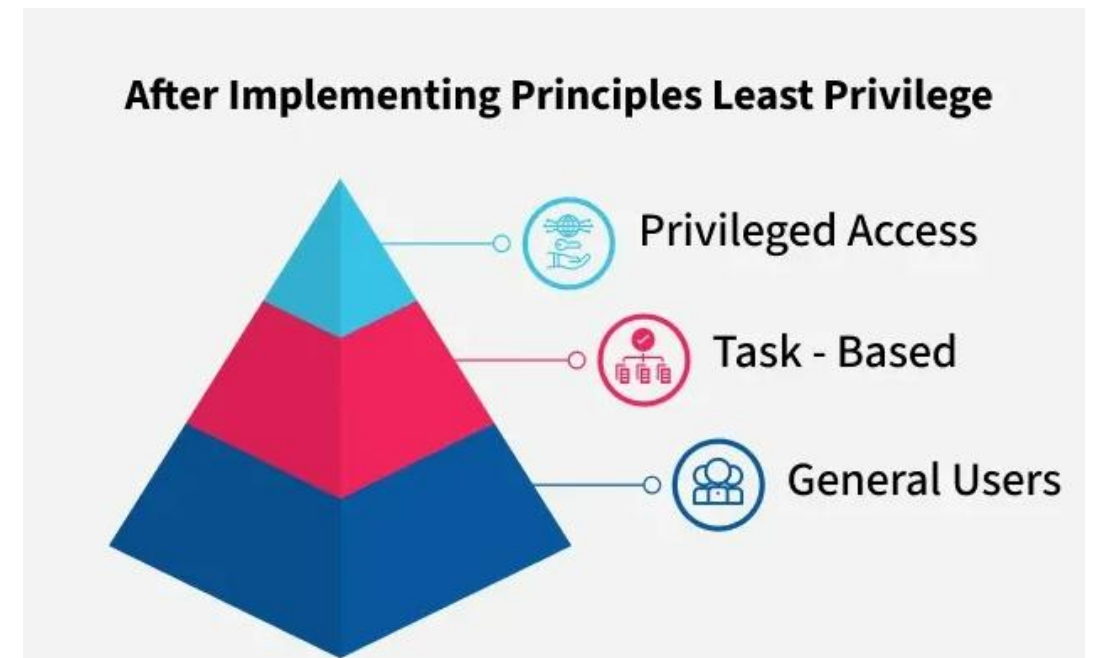
文件操作 进程管理 权限控制 网络通信

<code>open()</code>	<code>fork()</code>	<code>chmod()</code>	<code>socket()</code>
<code>read()</code>	<code>exec()</code>	<code>setuid()</code>	<code>connect()</code>
<code>write()</code>	<code>exit()</code>		<code>send()</code>
<code>close()</code>			



最小权限原则

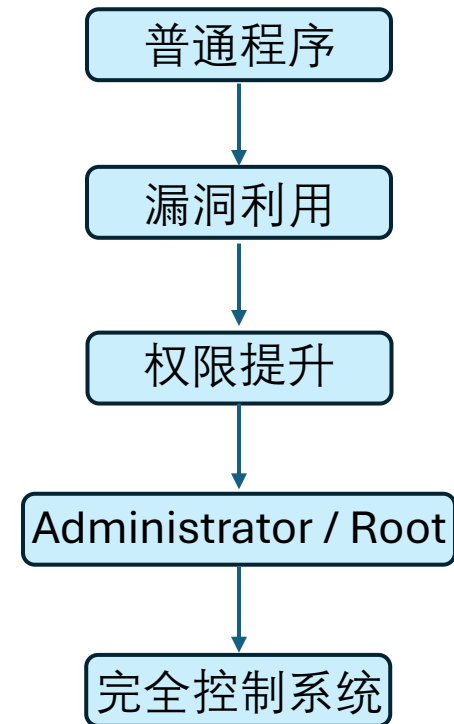
- 最小权限原则：仅向用户、系统进程或程序授予完成其当前任务所需的**最低限度访问权限**。
 - 只给予完成任务所需的最低权限
- 普通用户不应拥有管理员权限
- App不应默认访问全部数据



为什么木马喜欢管理员权限？

- 权限决定攻击能力
- **管理员权限**拥有操作系统的最高控制能力
 - 修改系统文件
 - 安装驱动 (Driver)
 - 修改注册表 (Registry)
 - 创建系统服务 (Service)
 - 关闭安全软件
 - 获取所有用户数据
 - 长期隐藏运行 (Persistence)

权限提升 是攻击核心



访问控制

- 访问控制包括**身份验证**和**授权**。
- 身份验证检验当前用户是否为系统内的合法主体（Subject）。
- 授权是针对系统内合法主体所允许的访问资源的操作限制。

项目	身份验证 (Authentication)	授权 (Authorization)
作用	确认“你是谁”	决定“你能做什么”
发生顺序	先发生	后发生
依据	用户凭证（密码、指纹等）	权限规则
结果	是否允许登录	是否允许访问资源

访问控制：
主体
客体
权限

权限：
读
写
执行

访问控制矩阵

	操作系统	会计程序	会计数据	保险数据	薪水数据	客体
Bob	rx	rx	r	—	—	
Alice	rx	rx	r	rw	rw	
Sam	rwX	rwX	r	rw	rw	
会计程序	rx	rx	rw	rw	r	
主体						

- e.g., Bob 主体对操作系统有读写权限。
Alice 主体对会计数据有读权限。
Sam 主体对会计程序有读，写，和执行权限。

Linux 权限模型

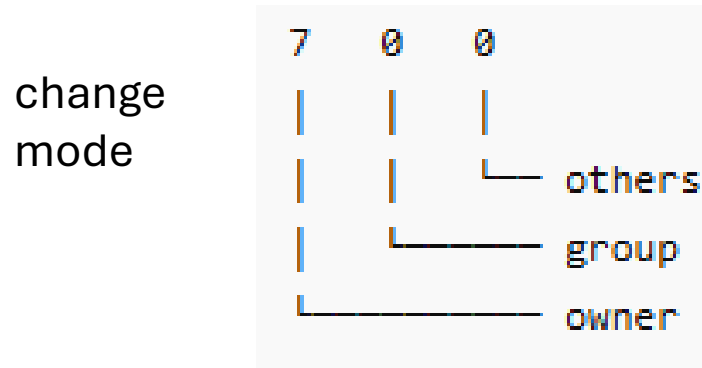
- Linux 系统下对应的文件权限: rwx权限模型
 - r = read (读)
 - w = write (写)
 - x = execute (执行)
 - e.g., chmod 700 file
- | 权限 | 数值 |
|-----|----|
| r-- | 4 |
| rw- | 6 |
| rwx | 7 |

权限	数值	含义
r--	4	只读
rw-	6	读写
rwX	7	完全权限
---	0	无权限

文件所有者具有读写执行完全权限

同组所有者 没有权限

其他用户 没有权限



Windows 权限控制

- 在Windows系统下，下载安装软件会遇到系统提示：
- 需要用户去确认是否继续
- 确认程序是否安全
- Windows 在做权限控制。



Windows 权限控制

- Windows系统下，用户分为管理员和普通用户

- 普通用户：

不能修改系统核心
不能安装驱动
不能修改系统配置
不能改其他用户数据
浏览网页
写文档
看视频
安装普通软件（部分）

管理员：

修改系统文件
安装驱动
修改注册表（Registry）
关闭杀毒软件
管理其他账户



可信操作系统 (Trusted OS)

- **可信操作系统**: 系统在安全上可信赖。即使用户或程序试图违规, 系统也不允许, 可以强制执行安全策略的操作系统。

普通操作系统:

用户自己决定权限
安全依赖用户配置
容易误操作

可信操作系统:

系统强制执行规则
用户无法绕过安全策略
默认更安全

- 保证系统在攻击下仍然可信运行

可信操作系统 (Trusted OS)

- 可信操作系统必须提供:

- 强制访问控制(MAC)
- 自主访问控制(DAC)
- 完全干预性
- 可信路径
- 日志

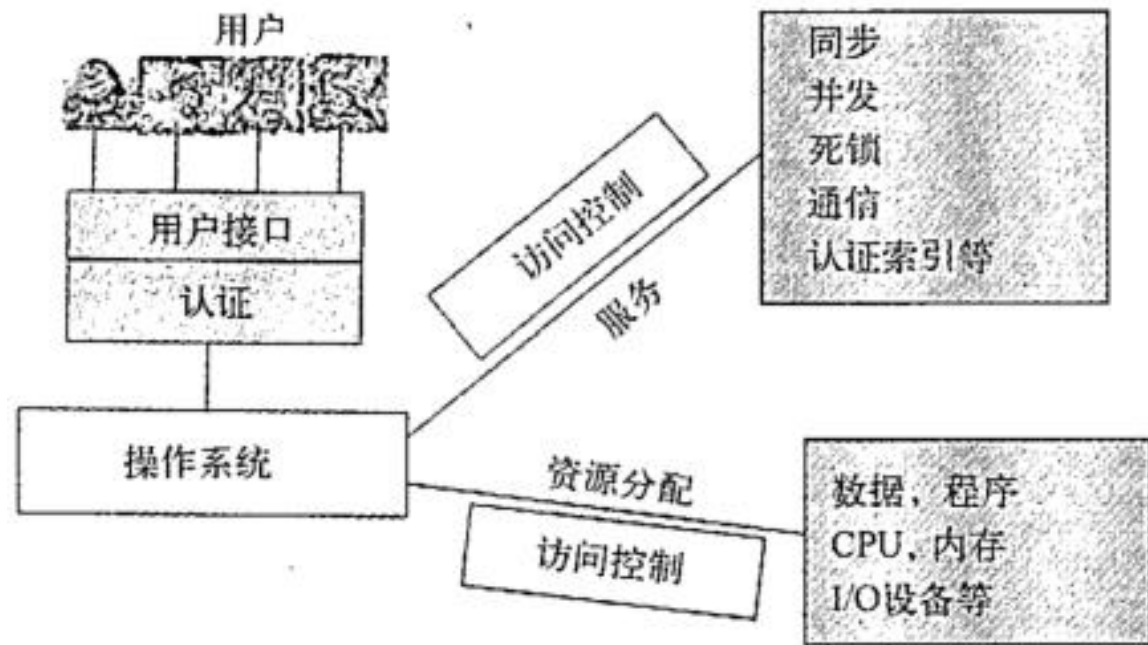
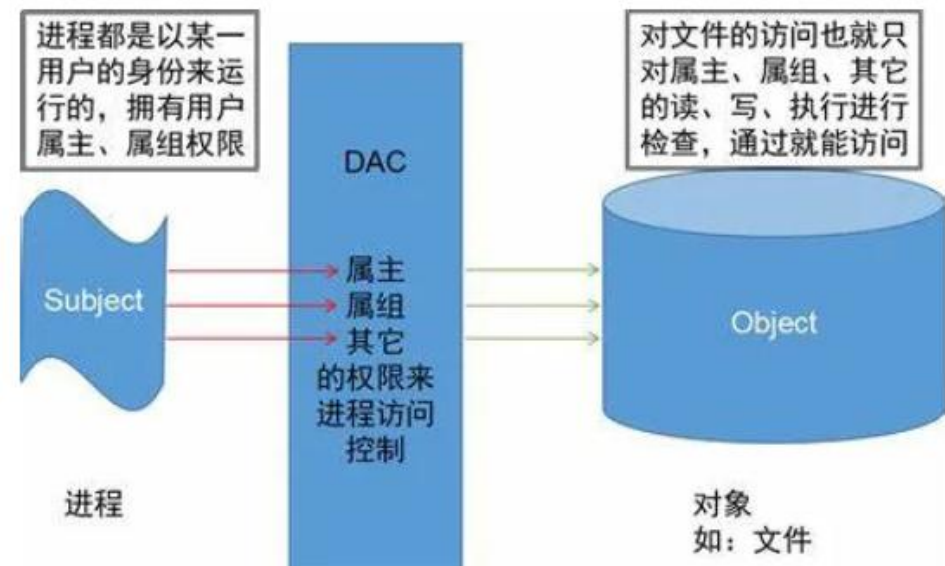


图 13.4 可信操作系统概要

自主访问控制 DAC

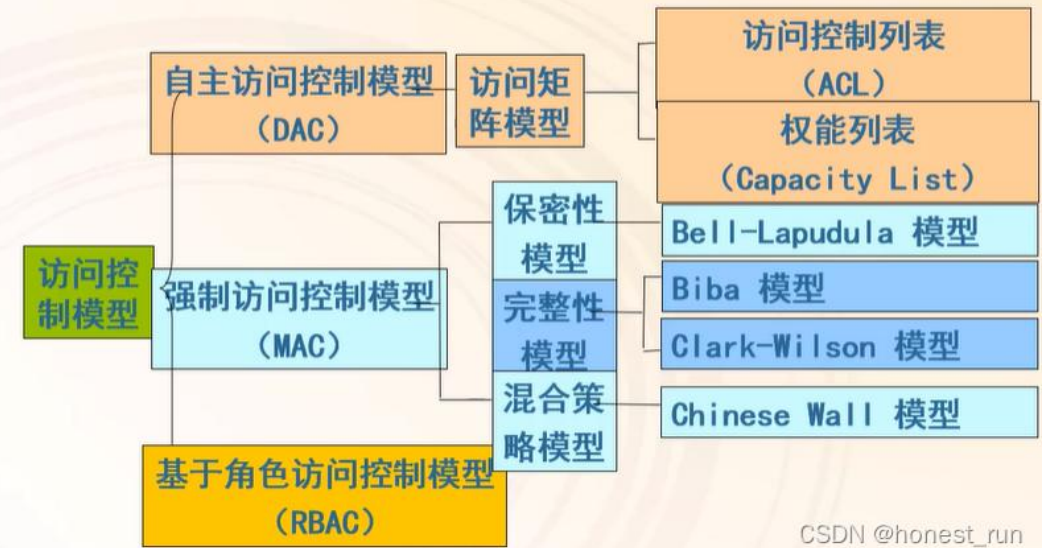
- 自主访问控制 DAC (Discretionary Access Control) : 控制权归**客体拥有者**本身所操控的访问控制模式
- DAC通过适用访问**控制矩阵**去描述主体和客体之间的权限分配关系。
- 案例： 文件系统权限分配
- 问题： 用户容易配置错误。



强制访问控制MAC

- 强制访问控制 (MAC Mandatory Access Control):
- 为了弥补DAC权限控制过于分散而诞生。由操作系统约束的访问控制，目标是**限制**主体或发起者访问或对对象目标执行某种操作的能力。
- 任何主体对任何对象的操作都根据一组授权规则进行测试。

访问控制模型分类



CSDN @honest_run

强制访问策略

- 强制访问控制系统根据主体和客体的敏感标记来决定访问模式，模式包括：
 - 不上读（NRU），主体不可读安全级别高于他的数据；
 - 不下读（NRD），主体不可读安全级别低于他的数据
 - 不上写（NWU），主体不可写安全级别高于他的数据。
 - 不下写（NWD），主体不可写安全级别低于他的数据。

机密性

BLP模型
不上读，不下写

Biba模型
不上写，不下读

完整性

DAC vs MAC

- 如果DAC和MAC同时施加在一个客体上时， **MAC具有优先权**。
- 例子：

如果Alice拥有一个标记有最高密级的文档，因为Alice是这个文档的拥有者，所以 she 可以将该文档访问模式设置为DAC。但是，对于仅拥有初等密级的Bob来说，无论无何设置，他都无法访问这个文档，因为他不满足MAC的要求。

强制访问控制MAC

- 强制访问控制、内存保护和剩余信息保护:
- 可信操作系统必须采取措施进行内存保护和访问控制来防止信息从一个用户泄露到另外一个用户。
- 操作系统为文件分配空间时，同样的空间可能之前被其他用户或进程使用，为了防止之前的数据被访问读取造成泄露，可信操作系统需要采取措施去**防止数据泄露**。

可信路径

- 可信路径：用户确认**正**在于可信系统通信。
- 使用可信路径防止用户被伪造界面欺骗。
- 你如何确定正在与真正的OS通信？

Recently added

 EarTrumpet

A

 Active Desktop Plus

 Adobe Creative Cloud

 Adobe Lightroom

 Adobe Photoshop Express

 Adobe XD

 Advanced Recovery Companion

 Alarms & Clock

 Amazon Prime Video for Windows

 Android Studio

 AppManager

 AudioWizard

B

 Bing Wallpaper

Productivity

Office

Excel

To Do 6

OneNote for...

PowerPoint

Microsoft Edge

Word

Photos

Films & TV

Camera

Explore





可信路径

- 为什么需要可信路径?

- 攻击者可能

伪造登录界面

伪造支付界面

伪造系统弹窗

窃取密码

아주대학교 통합인증

사용자 ID를 입력해주세요.

비밀번호를 입력해주세요.

로그인

[통합 ID 신청](#) | [ID/PW 찾기](#) | [시스템문의](#)

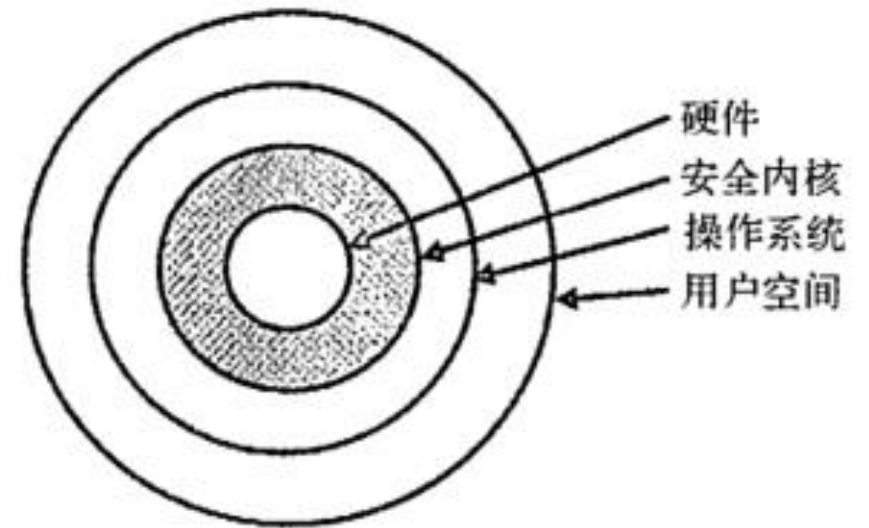
- 如何确认你正在与真正的系统通信而不是假的登录界面?
- 可信路径 = 系统证明“我是正版”的方式。

可信路径案例

- Windows 系统下，输入Ctrl + Alt + Delete
- 强制进入系统级登录界面：**Kernel** 管理的真实登录界面
- 这是Windows系统下的一个可信路径。
- 只有OS能回应这个快捷键。
- 可信路径为什么可信→ 可信路径直接连接可信计算基

可信计算基-TCB

- 可信计算基 (Trusted Computing Base) : 整个系统安全的基础所在。
- 即使TCB之外的所有设施被攻破, 整个可信操作系统依然是安全的。
 - **内核**: 操作系统的最底层, 负责同步、进程交互、消息传递、中断调度等。
 - **可信内核**: 用来进行安全处理的内核部分。
 - **安全内核**: 访问控制的理想位置。安全函数理想位置。



理想的可信计算基设计

为什么TCB越小越好

复杂性 (Complexity) 是安全最大的敌人。

- 可信计算基 (TCB) 包括：
 - Kernel (内核)
 - Security Mechanisms (安全机制)
 - Authentication Module (认证模块)
 - Hardware (关键硬件)
- TCB是整个可信操作系统的基础, TCB出错, 则系统失去可信性。
- TCB越小越安全：
 - 更容易验证
 - Bug更少
 - 攻击面更小

宏内核 vs 微内核

- 宏内核: Linux/Windows系统
 - 很多功能都在内核里, 包括驱动, 文件系统, 网络堆栈。
 - 内核内容越多, 越复杂, 安全性越弱
- 一处出现漏洞影响整个系统
- 微内核: 只保留核心功能 Qubes OS, MINIX 3
 - 只保留调度 (scheduling), 存储管理 (Memory Management) 和进程通信。
 - 简单, 容易验证, 攻击面小。

Rootkit 与 Kernel Attack

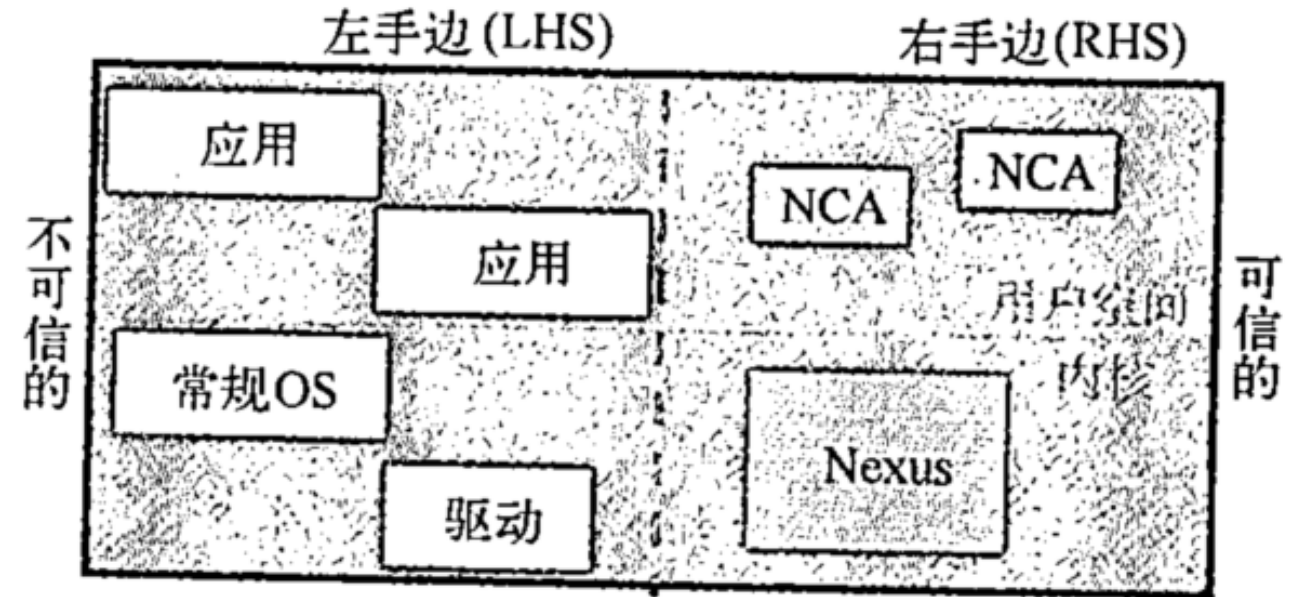
- **普通的恶意软件**：从应用层软件获取对应的普通用户权限。
 - 破坏能力有限
- **内核Rootkit**：从应用层软件获取权限→权限提升→进入内核模式→修改OS行为。
 - 目的不是感染系统，而是接管系统。隐藏自己的行为
- Sony的音乐CD为了防止盗版偷偷安装了Rootkit，隐藏文件和进程。
- 黑客利用了Rootkit漏洞攻击了用户。→导致Sony大规模召回。

Next-Generation SCB

- 传统的TCB包含内核，安全模块和验证模块，太大，复杂容易被攻击。
 - Kernel漏洞
 - Rootkit
 - Driver Attack
 - Supply Chain Attack
- 微软2002年提出了：下一代安全计算基（Next-Generation Secure Computing Base）

Next-Generation SCB

- 微软提出：
 - 在普通Windows 内部建立可信环境。
- 即使操作系统被攻击，关键数据仍然安全。
- 思想：不完全相信OS，创建一个受信任的安全环境保护敏感数据等。



进程隔离：防止进程间干扰

密封存储：通过防篡改硬件安全存储

安全路径：提供鼠标，键盘，显示器受保护路径

证明：允许对主体进行安全验证。

Next-Generation SCB

- 微软的NGSCB设计最终因为很多原因失败了。
 - 太复杂：要求CPU, TPM Driver, OS一起升级。
 - 兼容性差：很多软件不支持
 - 用户反感：其中包含了DRM数字版权 微软控制电脑。

NGSCB 思想

今天实现

TPM

Windows 11 TPM 2.0

Trusted Path

Secure Login

Secure Execution

SGX / TrustZone

Secure Boot

Verified Boot

Isolation

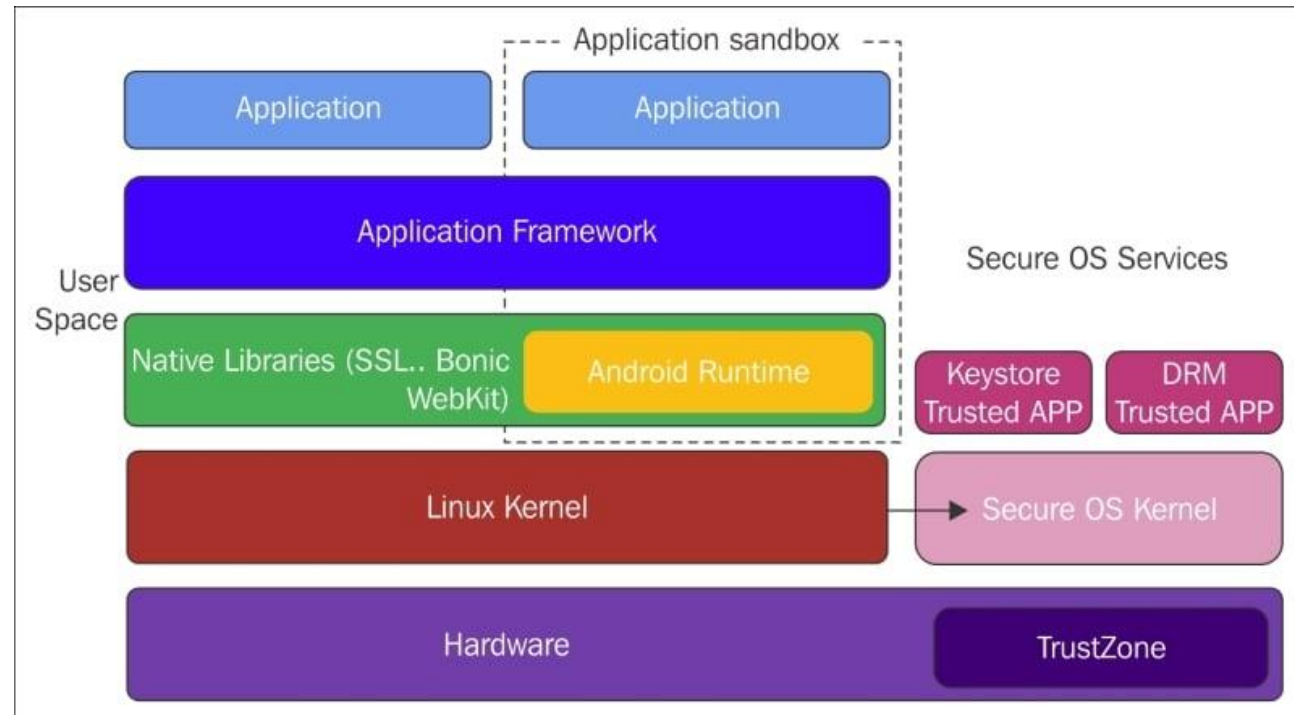
Sandbox

现代可信计算

- 现代可信计算的核心思想：少信任，多验证。
- 现代可信计算：
 - Hard Root of Trust （硬件信任根）
 - Secure Boot （安全启动）
 - Trusted Execution Environment （可信执行环境）
 - Virtualization-based Security （虚拟化安全）
 - Remote Attestation （远程证明）

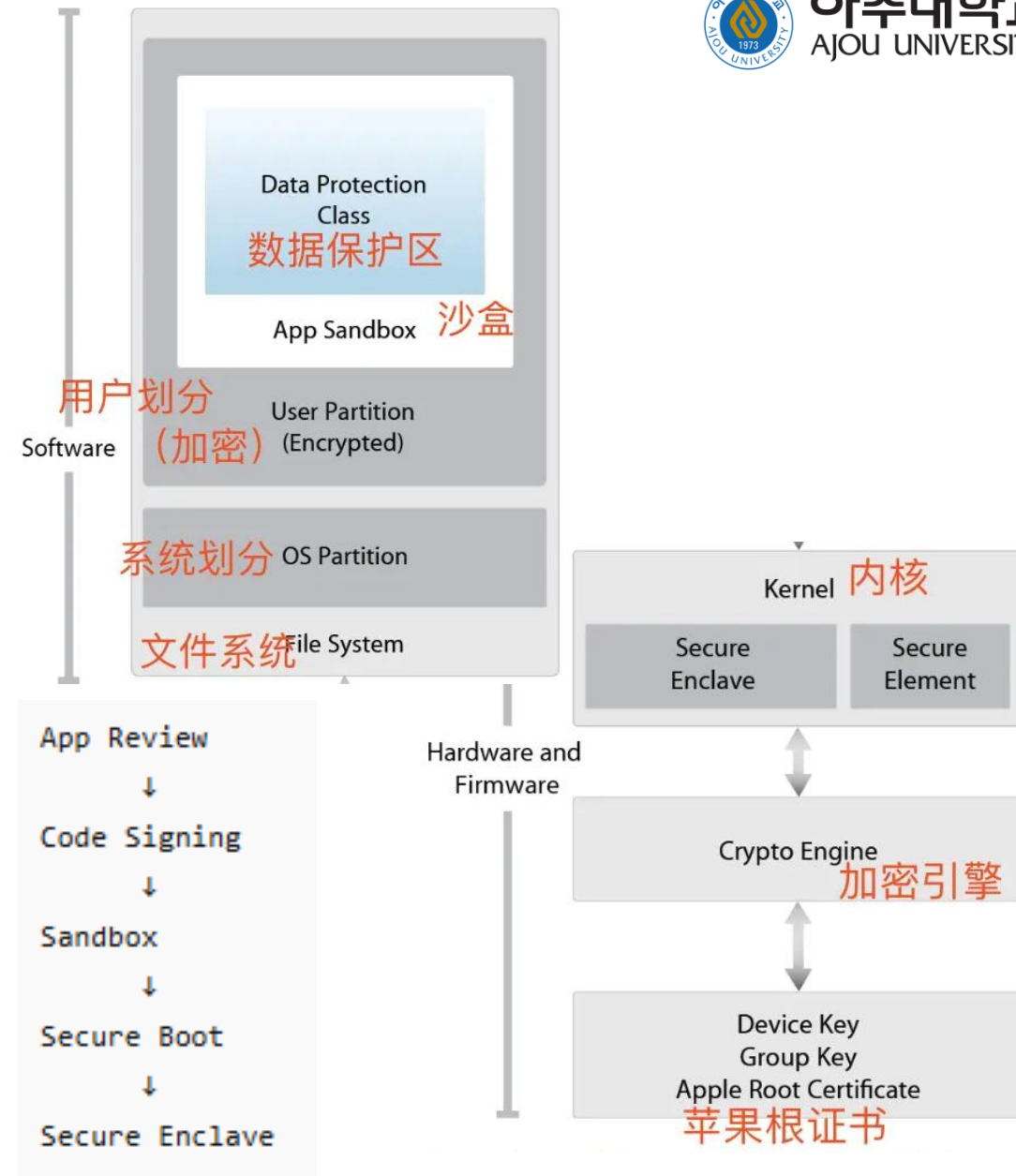
Android安全模型

- Android系统采用**默认不信任**应用
- 原则：
 - Sandbox Isolation (沙箱隔离)
 - Permission Control (权限控制)
 - Application Signing (应用签名)
 - Linux Kernel Security (Linux 内核保护)
 - Secure Authentication (安全认证)



iOS安全设计

- iOS系统采用默认不信任App，最小化系统攻击面
- 原则：
 - Sandbox Isolation (沙箱隔离)
 - Application Signing (应用签名)
 - Secure Boot (安全启动)
 - Hardware Root of Trust (硬件信任根)
 - Trusted Execution Environment (TEE)



Android vs iOS

Android

更开放

APK 可侧载

Google 审核较宽

灵活性高

攻击面更大

iOS

更封闭

默认禁止

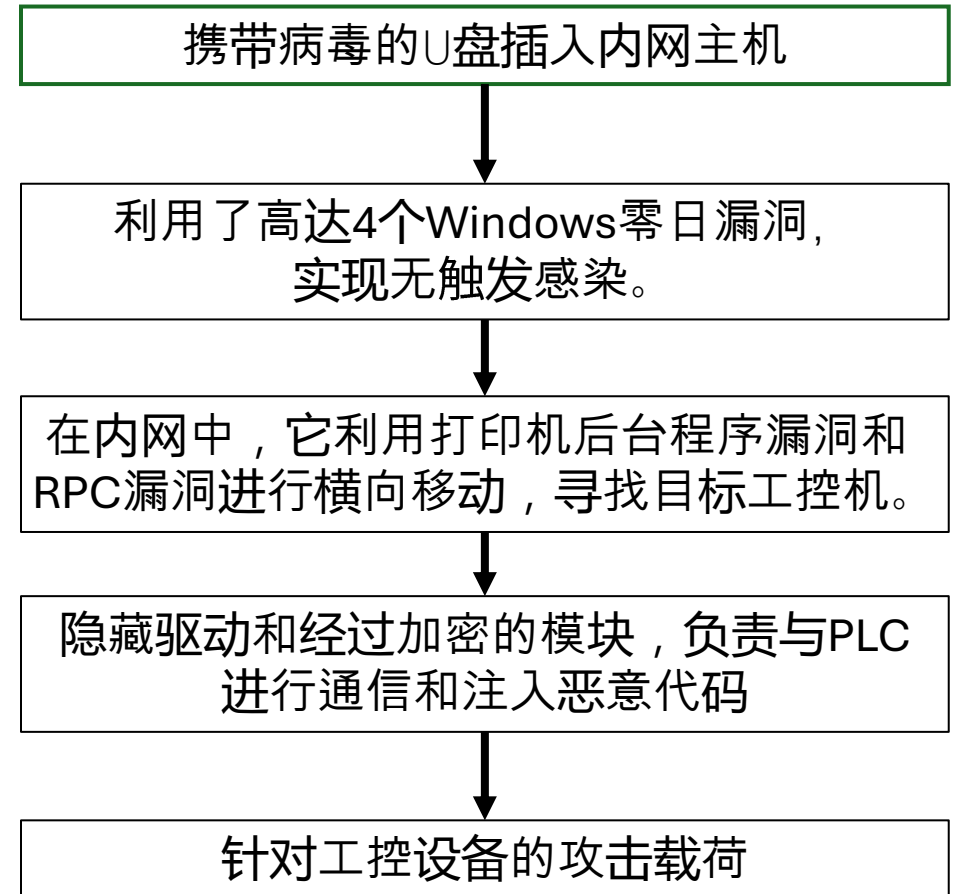
苹果审核严格

控制更强

攻击面更小

现代攻击案例

- Stuxnet（震网病毒）：利用计算机蠕虫病毒对伊朗核设施的铀浓缩离心机进行摧毁。
 - USB 感染
 - 系统漏洞利用
 - 权限升级
 - 内核Rootkit
 - 工业控制攻击



结语

- 密码学知识（对称密码，公钥密码，哈希函数与密码分析）
- 访问控制（认证，授权）
- 协议安全（通信网络安全）
- 软件安全（漏洞与缺陷，恶意软件，软件逆向工程）
- 操作系统安全

Thank you.